

Step 1: Data Preparation

This stage includes loading the dataset and applying data augmentation and transformations such as scaling, rotation, mirroring and cropping. Normalisation is also undertaken at this stage.

Imports: We import the necessary libraries for data transformations, loading datasets, and creating data loaders."

```
import torch
import torchvision.transforms as transforms
from torchvision.datasets import CIFAR10
from torch.utils.data import DataLoader, random_split
```

Transformations/augmentation and normalisation: We define a series of transformations to apply to each image..

- * **RandomResizedCrop(224):** Randomly crop the image to 224x224 pixels.
- * **RandomHorizontalFlip():** Randomly flip the image horizontally.
- * **RandomRotation(10):** Randomly rotate the image by ± 10 degrees.
- * **RandomVerticalFlip():** Randomly flip the image vertically.
- * **ToTensor():** Convert the image to a PyTorch tensor.
- * **Normalize(mean, std):** Normalize the tensor using the given mean and standard deviation."

```
# Define data augmentation and normalisation transformations
transform = transforms.Compose([
    transforms.RandomResizedCrop(224),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),
    transforms.RandomVerticalFlip(),
    transforms.ToTensor(),
    transforms.Normalize((0.485, 0.456, 0.406), (0.229, 0.224, 0.225))
])
```

Dataset Loading: We load the CIFAR-10 dataset, applying the transformations defined above, to each image."

```
# Load CIFAR-10 dataset
dataset = CIFAR10(root='./data', train=True, download=True, transform=transform)
```

Files already downloaded and verified

Dataset Splitting: We split the dataset into training, validation, and test sets...

- * **train_size:** 80% of the data for training.

- * **val_size:** 10% of the data for validation.

- * **test_size:** The remaining 10% for testing.

```
# Split dataset into training, validation, and testing sets
train_size = int(0.8 * len(dataset))
val_size = int(0.1 * len(dataset))
test_size = len(dataset) - train_size - val_size

train_dataset, val_dataset, test_dataset = random_split(dataset, [train_size, val_size, test_size])
```

Data Loaders: We create data loaders for each dataset split..

- * **train_loader:**** Loads training data in batches of 32 and shuffles the data.

- * **val_loader and test_loader:**** Load validation and test data in batches of 32 without shuffling.

```
# Create data loaders
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=32, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)
```

Step 2: Model Loading

This stage uses a pre-trained network.

Pre-trained Model: We load a pre-trained ResNet18 model. Pre-trained models are models that have been previously trained on large datasets (e.g., ImageNet) and can be fine-tuned for specific tasks.

```
import torchvision.models as models

# Load a pre-trained ResNet18 model
model = models.resnet18(pretrained=True)
```

Model Modification: We replace the final fully connected layer to match the number of classes in CIFAR-10 (10 classes which are Airplane, , Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship, Truck).

*** num_fts** holds the number of input features to the final layer, which we use to define the new fully connected layer.**

```
# Modify the final layer to match the number of classes in CIFAR-10
num_fts = model.fc.in_features
model.fc = torch.nn.Linear(num_fts, 10)
```

Step 3: Training Script

Utilise the GPU for training, with an option to choose different models.

Device Selection: We check if a GPU is available and set the device accordingly. The model is moved to the selected device (GPU if available, otherwise CPU).

```
import torch.optim as optim
import torch.nn as nn
from torch.cuda import is_available
# Check for GPU
device = torch.device("cuda:0" if is_available() else "cpu")
model = model.to(device)
```

Loss and Optimiser: We define the loss function (CrossEntropyLoss) and the optimiser (Stochastic Gradient Descent) with a learning rate of 0.001 and momentum of 0.9.

```
# Loss function and optimizer
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=0.001, momentum=0.9)
```

Training Loop: We train the model for normally 10 epochs but for testing purposes we will set this to 2:

- * `model.train():` Set the model to training mode.
- * Iterate over batches of data from `train_loader`
- * Move inputs and labels to the selected device (GPU/CPU).
- * Zero the gradients, perform forward pass, compute loss, perform backward pass, and update the weights
- * Accumulate the running loss and print the average loss per epoch.

NOTE: The duration of the training loop can vary significantly depending on your hardware, it can take somewhere between 10 minutes and 6 hours. You could leave it running overnight

```
#Training loop
num_epochs = 2

for epoch in range(num_epochs):
    model.train()
    running_loss = 0.
    for inputs, labels in train_loader:
        inputs, labels = inputs.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    print(f"Epoch {epoch+1}/{num_epochs}, Loss: {running_loss/len(train_loader)}")
```

Step 4: Data Loading and Batching

Data loaders for training, validation, and testing were already created in Step 1. These loaders handle the batching and shuffling of data as required.

Step 5: Saving and Loading Checkpoints

Save the trained model and load it for inference. Inference is the phase in development where the capabilities learned during training is put to work.

*** Save Checkpoint:** Save the model's state dictionary (parameters) to a file `*(model.pth)`

*** Load Checkpoint:** Load the state dictionary from the file and set the model to evaluation mode `*(model.eval())`

```
# Save the model checkpoint
torch.save(model.state_dict(), 'model.pth')

# Load the model checkpoint
model.load_state_dict(torch.load('model.pth'))
model.eval()
```

Step 6: Inference with Transformations

Load Image: Load an example image from a URL. You must replace the URL shown below with one of classes to test it for inference.

Instructions on how to find an images URL are [[here](https://www.wikihow.com/Get-the-URL-for-Pictures)](https://www.wikihow.com/Get-the-URL-for-Pictures)"

```
from PIL import Image
import requests
from io import BytesIO

# Load an example image
# a few examples to test inference could be
# https://cdn.britannica.com/79/232779-050-6B0411D7/German-Shepherd-dog-Alsatian.jpg
# https://static1.simpleflyingimages.com/wordpress/wp-content/uploads/2023/08/emirates-a380-with-new-livery-by-vincenzo-pace-from-sf.jpg
# https://cdn.mos.cms.futurecdn.net/39CUYMP8vJqHAYGVzUghBX-1200-80.jpg
# https://i.natgeofe.com/n/548467d8-c5f1-4551-9f58-6817a8d2c45e/NationalGeographic_2572187_square.jpg

url = "https:replaceme.jpg"
response = requests.get(url)
img = Image.open(BytesIO(response.content))
```

Transformations for Inference: Apply similar transformations to the image as used during training...**

- * Resize to 256x256 pixels.**
- * Center crop to 224x224 pixels.**
- * Convert to tensor.**
- * Normalise using the same mean and standard deviation.**

Batch Dimension: Add a batch dimension to the image tensor and move it to the selected device.

```
# Apply the same transformations as used in training
transform_inference = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize((0.485, 0.456, 0.406), (0.229, 0.224, 0.225))
])

img_t = transform_inference(img)
img_t = img_t.unsqueeze(0).to(device) # Add batch dimension and move to GPU
```

Inference: Perform inference on the image..**

- * Set the model to evaluation mode `**(model.eval())`**
- * Disable gradient computation `**(torch.no_grad())`**
- * Forward pass the image through the model to get the output.**
- * Use `**torch.max**` to get the predicted class from the output.**
- * Print the predicted class - the specific class predicted by the model will depend on the content of the image you use for inference.**
- * If `"Predicted class = 0"` it means Airplane**
- * `#0 Airplane #1 Automobile #2 Bird #3 Cat #4 Deer #5 Dog #6 Frog #7 Horse #8 Ship #9 Truck`**

```
# Inference
model.eval()
with torch.no_grad():
    output = model(img_t)
    _, predicted = torch.max(output, 1)
    print(f'Predicted class: {predicted.item()}')
```